أكاديمية سدايا
SDAIA Academy

SDAIA
الهيئة السعودية للبيانات والذكاء الاصطناعي
Saudi Data & AI Authority

SDAIA ACADEMY · APPLIED GENERATIVE AI

Retrieval you can measure

Day 2 · Monday 3 August 2026 · Musa Ibn Rashid

Everyone builds RAG. Almost nobody measures it.

PRESENTER NOTES

Today is the longest day of the week and the one with the highest ceiling, so pace yourselves.

By half past two you will not just have built retrieval over your own documents — you will have a number that says how good it is.

That number is the difference between "it looked right in the demo" and something you would put in front of a director.

THE WEEK

Where we stand

SUNDAY	~~From token to typed output — the model as a reliable function.~~
MONDAY	Retrieval you can measure — RAG, hybrid search, and a golden set that scores it.
TUESDAY	Tools and agents — the model requests, your code executes.
WEDNESDAY	Production and the numbers — latency, caching, and what this costs per user.
THURSDAY	Break it, then defend it — prompt injection, red teaming, and your demo.

> **There is no such thing as a dumb question.** If you are wondering, someone else is too — ask.

PRESENTER NOTES

Point at Sunday and remind them what they already have: a function that returns typed JSON their code can branch on.

Today is the longest day of the week and the one with the highest ceiling. Say that out loud so nobody is surprised at two o'clock.

Everything you build today becomes a tool inside tomorrow's agent, so falling behind here is expensive.

أكاديمية سدايا SDAIA Academy

SDAIA

TODAY

How the day runs

9:15 – 10:05	**Block 1 · Choosing an architecture**	50 min
10:05 – 10:20	Break	
10:20 – 11:00	Block 2 · The pipeline, stage by stage	40 min
11:00 – 11:20	Break	
11:20 – 12:00	Block 3 · Chunk Lab, then Notebook 2 Part A	40 min
12:00 – 1:00	Lunch	60 min
1:00 – 1:50	Block 4 · Hybrid, re-ranking, query rewriting	50 min
1:50 – 2:00	Break	
2:00 – 2:30	Block 5 · Golden set, scoring, project teams	30 min

PRESENTER NOTES

Same grid every day, so once they know it they stop asking when lunch is.

Part A of the notebook lands before lunch on purpose, so lunch absorbs anyone who falls behind.

The two ceiling blocks are after lunch. Protect your energy for them.

DAY 2

What you will be able to do by 2:30

01 Explain why the model cannot answer questions about your documents, and what fixes it.

02 Choose between RAG and an agent, and defend the choice in one sentence.

03 Build the pipeline end to end: ingest, chunk, embed, store, retrieve, answer with citations.

04 Pull text out of a scanned Arabic page with a vision model and feed it into that pipeline.

05 Combine keyword and vector search, then re-rank what comes back.

06 **Score your retrieval against a golden set you wrote yourself.**

PRESENTER NOTES

Six today, and the last one is the one that earns this course its name.

Almost nobody who builds retrieval does number six, which means almost nobody can tell you whether their last change helped or hurt.

By this afternoon you will be able to say "my retrieval went from six out of ten to nine out of ten", and mean it.

أكاديمية سدايا
SDAIA Academy

SDAIA

YESTERDAY · QUESTIONS TO THE ROOM

Three questions before we start

01 Why does the same sentence cost more in Arabic than in English?

02 You need extraction that your code will parse. What temperature, and why?

03 What does a response schema buy you that a well-written prompt does not?

> No slide of answers. Ask three different people, and make them say it out loud.

PRESENTER NOTES

Do not answer these yourself. Pick three people who did not speak much yesterday and give them the floor.

If the room struggles with the third one, spend two minutes back on yesterday's schema slide before you move on, because everything today assumes it.

Then close by saying: yesterday the model became a reliable function; today we give it something to be reliable about.

أكاديمية سدايا
SDAIA Academy

SDAIA

PART ONE

Choosing an architecture

RAG or an agent? Get this wrong and you spend three weeks building the wrong thing.

PRESENTER NOTES

Before any code, the decision. This is the conversation you will have back in your organisation, probably in a meeting where nobody has built either one.

The good news is the decision rule fits on a single slide, and we get to it in about fifteen minutes.

THE FLOOR

It has never seen your documents

The model learned from public text, up to a cut-off date. Your leave policy, your circulars, your internal manual — none of it was there.

Ask anyway and you do not get "I don't know". You get a **plausible invention**, because likely text is the only thing it produces.

Three ways out

- **Paste it in** — works until the document is bigger than the context window. Which it is.
- **Fine-tune** — weeks, expensive, and it teaches style rather than facts.
- **Retrieve it** — find the three relevant paragraphs, put those in the prompt. **This one.**

> RAG is not clever. It is "look it up first, then answer" — the thing you would do yourself.

PRESENTER NOTES

Ask the room: if I asked you about a policy you had never read, what would you do? Somebody will say "go and read the relevant part". That is the entire idea.

The reason we do not simply paste the whole document is yesterday's context window slide, and it is also a cost argument — every token you paste, you pay for on every single call.

CLICK EACH STEP

RAG in four steps

STEP 1
Index, once

STEP 2
Retrieve

STEP 3
Augment

STEP 4
Generate

> Only step 1 happens in advance. Steps 2 to 4 run on every question, in that order, every time.

PRESENTER NOTES

Click these one at a time as you talk, so the room watches the pipeline assemble instead of arriving at a finished diagram.

The point to hammer is that step three is string concatenation. There is no magic in retrieval-augmented generation — all the intelligence is in choosing which chunks go in.

And that choice is exactly what we spend this afternoon improving, and then measuring.

THE OTHER ARCHITECTURE

An agent is a loop

01 Give the model a goal and a list of tools.

02 It decides which tool to call, and with what arguments.

03 **Your code** runs the tool and hands the result back.

04 Repeat until it says it is done — or until your step cap stops it.

RAG has one fixed path. An agent chooses its own — that is the whole difference.

What that buys, and costs

Buys: it can take several steps, react to what it finds, and *do* things rather than only answer.

Costs: every step is another call. Unpredictable path, unpredictable bill, and much harder to test.

PRESENTER NOTES

Tomorrow is entirely about this loop, so today all I want is that you can tell the two shapes apart.

Say the third step firmly, because it is the most misunderstood thing in this field: the model never runs anything. It emits a request, and your code decides whether to honour it.

CHOOSING

RAG or agent?

	RAG	AGENT
Shape	One fixed path, every time	A loop that picks its own path
Good at	Answering from your documents	Multi-step tasks, live data, taking action
Calls per question	One	Three to ten, and you cannot be sure
Build time	Days	Weeks, most of it on guardrails
Fails by	Retrieving the wrong chunk	Looping, burning money, acting wrongly

> **The rule:** if the answer is *in a document*, use RAG. If it needs a *decision or an action*, consider an agent. Start with the simplest thing that works, then add.

PRESENTER NOTES

Read the bottom line twice, because it is the advice most likely to save someone in this room a wasted month.

Nearly every "we need an agent" I meet is a retrieval problem with better marketing.

And notice the last row: RAG fails by being unhelpful, an agent fails by doing something. Those are very different risks to carry into production.

أكاديمية سدايا
SDAIA Academy

SDAIA

PART TWO

The pipeline, stage by stage

1 · Ingestion	Get clean text out of whatever you were given — and keep the source.
2 · Chunking	Cut it into pieces small enough to retrieve, large enough to mean something.
3 · Embedding	Turn each chunk into a vector — meaning as coordinates.
4 · Storage	Put the vectors somewhere you can search by nearness.
5 · Retrieval	Embed the question, pull the nearest chunks, hand them to the model.

PRESENTER NOTES

Five stages, and every one of them has a way of quietly ruining the stage after it.

Most broken retrieval systems I have seen were broken in stage one or two, while the team spent their time tuning stage five.

STAGE 1

Ingestion: boring, and decisive

Clean it

- Strip headers, footers, page numbers, watermarks.
- Keep the structure that carries meaning: headings, lists, tables.
- Fix the encoding before you look at anything else.

Keep the metadata

Every chunk travels with `source` and `page`, from the very first step.

Without it you cannot cite. Without citations nobody in a government context will trust the answer — and they are right not to.

Adding it later means re-ingesting everything.

> Garbage in, confidently-worded garbage out.

PRESENTER NOTES

This is the least glamorous stage and the one that decides whether the rest of the pipeline can work at all.

The metadata point is the expensive one to learn late. Carry source and page from the first line of code, because retrofitting means re-processing every document you have.

In the notebook this part is written for you, but read it, because you will be doing exactly this to your own documents this afternoon.

STAGE 1 · THE APPLIED BIT

A scanned page has no text layer

Half the Arabic PDFs in a government archive are photographs of paper. Open one in Python and you get an empty string back.

So make the vision model a **stage in the pipeline**: page image in, text out, into the same chunker as everything else.

Why not classical OCR

- Handles Arabic script and layout far better than a plain OCR engine.
- Reads a table as a table, not as scrambled lines.
- You can instruct it: "transcribe exactly, do not summarise".

Cost: one call per page, once, at index time. Never per question.

> This is not a modality tour. It is a pipeline stage that solves a problem you will meet this week.

PRESENTER NOTES

Ask how many people have a stack of scanned Arabic PDFs somewhere. Most hands go up, and this is the slide they will remember in six months.

Say the instruction out loud — transcribe, do not summarise — because a vision model asked vaguely will hand you a helpful summary and silently lose the detail you needed.

And it happens once, at index time. If you find yourself calling a vision model per question, something has gone wrong.

STAGE 1 · CODE

Page image in, text out

```
import pathlib
from google.genai import types

page = pathlib.Path("scan_p03.png").read_bytes()

PROMPT = (
    "Transcribe this page exactly as written. "
    "Keep Arabic text in Arabic. Keep table rows "
    "on separate lines. Do not summarise."
)

resp = client.models.generate_content(
    model="gemini-2.5-flash-lite",
    contents=[
        types.Part.from_bytes(data=page,
                              mime_type="image/png"),
        PROMPT,
    ],
)

docs.append({"text": resp.text,
             "source": "circular_2024.pdf",
```

Three things matter here

- The image is a `Part`; the instruction is plain text in the same list.
- "Do not summarise" is not politeness — without it you lose detail.
- The result is appended in **exactly the same shape** as a text-extracted page, so nothing downstream knows the difference.

PRESENTER NOTES

Look at the last three lines. The scanned page joins the corpus in the same dictionary shape as every other page, and that is what makes this a pipeline stage rather than a party trick.

In the notebook this runs against a real scanned page and Arabic comes back as Arabic. Watch the room when it does.

One caution to say out loud: it can mis-transcribe a digit, so for anything financial or legal a human still checks the page.

STAGE 2

Chunking: the decision nobody documents

Size

Too small and the answer is cut in half. Too big and you retrieve three paragraphs of noise around one useful line.

~500

tokens. A starting point, not a law.

Overlap

Repeat the tail of the previous chunk, so a sentence that spans a boundary survives whole in one of them.

~10%

of chunk size. Costs storage, saves answers.

Structure-aware

Split on headings, then paragraphs, then sentences — never blindly on character count.

Keep a heading attached to the text it introduces.

> Start at ~500 tokens with ~10% overlap. Then **measure**, and tune. By 2:30 you will have the tool to do that.

PRESENTER NOTES

You will find two different numbers in the wild — some decks say two to five hundred words, others say five hundred to a thousand tokens. Pick one and be consistent; I am using tokens all week.

Say plainly that five hundred is a starting point, not a result. The right chunk size for your documents is whichever one scores best on your golden set.

The third column is where most of the win is, and it costs nothing.

STAGE 2 · THE FAILURE

What a blind split does

Split every 500 characters:

```
6.2 Annual leave entitlement by grade
Grade 9  – 21 days
Grade 10 – 24 da
```

CHUNK BOUNDARY

```
ys
Grade 11 – 30 days
Grade 12 – 30 days + 5
```

The second chunk is a table with no heading and no first rows. Retrieved on its own, it means nothing.

What that costs you

- "How much leave does grade 11 get?" retrieves a fragment with no context.
- The model sees "Grade 11 — 30 days" with no idea it is about leave.
- It answers anyway. Fluently. Possibly about something else entirely.

Fix: split on the heading boundary, keep the table whole, and repeat the heading into the chunk.

PRESENTER NOTES

This is the most common defect in a first retrieval build, and it stays invisible until somebody asks exactly the wrong question in front of somebody important.

Notice what got orphaned: the heading. The word "leave" appears only there, so a chunk without it will never match a question about leave, no matter how good your embedding model is.

Chunk Lab this morning is this exact problem, with a real document and your own hands on the boundaries.

STAGE 2 · CODE

The chunker

```python
def chunk(text, size=500, overlap=50):
    # Paragraphs first — structure before size.
    paras = [p.strip() for p in text.split("\n\n") if p.strip()]
    chunks, current = [], ""

    for para in paras:
        # Room left? Keep filling this chunk. (~4 chars/token)
        if len(current) + len(para) < size * 4:
            current += para + "\n\n"
        else:
            chunks.append(current.strip())
            tail = current[-overlap * 4:]  # carry the overlap
            current = tail + para + "\n\n"

    if current.strip():
        chunks.append(current.strip())
    return chunks

# Every chunk keeps where it came from.
records = [{"text": c, "source": d["source"], "page": d["page"]}
           for d in docs for c in chunk(d["text"])]
```

Read the order

- Paragraphs first, size second. Structure wins.
- The highlighted line is the overlap: the tail of one chunk starts the next.
- The final comprehension is where `source` and `page` survive.

In the notebook the loop body is a `TODO`. You write it.

PRESENTER NOTES

This is twelve lines and it is most of what a chunking library does for you. Writing it once means you can debug it when it misbehaves.

The overlap line is the one people delete because it looks wasteful, and then they lose every answer that straddles a boundary.

After you write it, print three chunks immediately and check nothing was cut mid-word. That habit will save you an afternoon.

STAGE 3

Meaning as coordinates

An embedding turns text into a list of numbers — a point in space — placed so that **similar meanings land near each other**.

"annual leave" ↔ "vacation days"

Not one word in common. Nearly the same point.

"annual leave" ↮ "annual budget"

One word in common. Far apart.

What follows from that

- Search stops being about words and starts being about meaning.
- It works across languages — an Arabic question can find an English chunk.
- Chunks are embedded once; the question is embedded on every query.
- `gemini-embedding-001` is free, which is what makes today possible at zero spend.

PRESENTER NOTES

Give this a beat, because it is where yesterday's mechanics pay off. The system is not matching words, it is matching positions in a space the model learned.

The cross-language point matters more here than anywhere else: a question typed in Arabic can retrieve the English policy that answers it, and that is genuinely useful in this building.

Flag the cost line too — embeddings are free on this tier and generation is not, and that shapes how you design.

Embed, then store

```python
import chromadb, time
from google.genai import types

db = chromadb.Client()
col = db.get_or_create_collection("policies")

def embed(texts, task="RETRIEVAL_DOCUMENT"):
    out = []
    # Batch and sleep — the free tier rate-limits.
    for i in range(0, len(texts), 32):
        r = client.models.embed_content(
            model="gemini-embedding-001",
            contents=texts[i:i + 32],
            config=types.EmbedContentConfig(task_type=task),
        )
        out += [e.values for e in r.embeddings]
        time.sleep(1)
    return out

col.add(
    ids=[f"c{i}" for i in range(len(records))],
    documents=[r["text"] for r in records],
    embeddings=embed([r["text"] for r in records]),
    metadatas=[{"source": r["source"]
```

Notes

- `task_type` differs for documents and for queries — the wrong one quietly costs accuracy.
- The `sleep` is not decoration. You will hit 429s today.
- The highlighted lines are your citations, surviving into the store.

Where else it can live

Chroma local, zero setup — what we use. **Pinecone** managed. **Weaviate** open source, hybrid built in. **pgvector** if you already run Postgres.

PRESENTER NOTES

Two things will bite you here and both are visible in the code. The first is the rate limit, which is why we batch and sleep. The second is re-running this cell and duplicating the collection.

The task type distinction is subtle and real: documents and queries are embedded slightly differently, and mixing them degrades results in a way that is very hard to notice.

On the database choice — start with Chroma. Nobody was ever fired for prototyping locally.

STAGE 5

Top-k: coverage against noise

Embed the question, ask the store for the *k* nearest chunks, paste them into the prompt.

k = 1 — one shot. Miss, and the answer is gone.

k = 3–5 — the working range for most document sets.

k = 20 — the answer is probably in there, buried in nineteen distractions you also pay for.

More is not better

- Every extra chunk is tokens on every single call.
- Irrelevant context actively pulls the answer off course.
- The fix for "the right chunk isn't in my top 3" is **better retrieval**, not a bigger k.

Unless you re-rank. Then you retrieve twenty on purpose — that is coming up.

PRESENTER NOTES

The instinct when retrieval misses is to raise k, and it feels like it works, because the right chunk is now somewhere in the pile.

But you have made every call more expensive and given the model more chances to answer from the wrong paragraph.

Hold that thought, because in twenty minutes I am going to tell you to retrieve twenty on purpose — the difference is that sixteen of them get thrown away.

أكاديمية سدايا
SDAIA Academy

SDAIA

PART THREE · THE CEILING

Making it actually good

You have working RAG. Now: where it fails, and what to do about it.

PRESENTER NOTES

Everything up to here gets you a demo that works on the questions you happened to try.

The next hour is what separates that from something you would let a colleague use, and it is the part of today that earns the word "applied".

THE CEILING

Where meaning-based search falls over

The query

`SDAIA-F-CRS-201-01-V1`

What vector search returns

Chunks about training programmes, course catalogues, form templates. All *topically* near. None of them the document.

That string has no meaning to embed. It is an **identifier**, and identifiers match on characters, not semantics.

Same failure, everywhere

- Document and form numbers
- Acronyms the model never learned
- People's names and place names
- Article and clause numbers — المادة 12
- Product codes, error codes

These are exactly the queries a professional user types.

PRESENTER NOTES

This is the slide where the room goes quiet, because everyone recognises that query. It is what a real employee types when they know precisely which document they want.

An embedding of a reference number is close to meaningless — the model never learned that string, so it lands somewhere arbitrary in the space.

The fix is not a better embedding model. It is admitting that keyword search was right about something, and that is exactly what we do next.

THE CEILING

Hybrid retrieval: run both

BM25 — keyword

Decades old, boring, unbeatable at exact strings. Scores a chunk on how often the query's rare words appear in it.

Finds `SDAIA-F-CRS-201-01-V1` instantly. Cannot connect "leave" to "vacation".

Vector — meaning

Finds the paragraph that answers the question even when it shares no words with it.

Connects "leave" to "vacation". Cannot find an ID.

> Normalise both score lists, add them with a weight, sort. **Two searches, one ranked list** — each covering the other's blind spot.

PRESENTER NOTES

The insight is that these two methods fail in opposite directions, and that is exactly when combining them is worth the extra complexity.

Normalising matters: BM25 scores and cosine similarities live on completely different scales, so adding them raw lets one drown the other.

Start the weight at fifty-fifty. If your users type a lot of reference numbers, push it towards keyword — and then measure whether that actually helped.

THE CEILING · CODE

Hybrid search

```python
from rank_bm25 import BM25Okapi
import numpy as np

# Built once, over the same chunks as the vector index.
bm25 = BM25Okapi([r["text"].split() for r in records])

def norm(x):
    # Squash to 0..1 so two score scales can be added.
    x = np.array(x, dtype=float)
    return (x - x.min()) / (np.ptp(x) + 1e-9)

def hybrid_search(query, k=5, alpha=0.5):
    kw  = norm(bm25.get_scores(query.split()))
    qv  = embed([query], task="RETRIEVAL_QUERY")[0]
    vec = norm(vector_scores(qv))   # same order as records

    score = alpha * vec + (1 - alpha) * kw

    top = np.argsort(score)[::-1][:k]
    return [records[i] for i in top]
```

The one line that matters

The highlighted line is the whole idea. `alpha=1.0` is pure vector search; `alpha=0.0` is pure keyword.

In the notebook, that line is your `TODO`.

Then re-run the query that failed two slides ago and watch it come back first.

PRESENTER NOTES

Everything on this slide is plumbing except one line, and that line is a weighted average.

The moment in the notebook is when the reference-number query that returned nothing useful comes back at rank one. Wait for that before you move on.

And do not tune alpha by feel. Tune it against the golden set you are about to write.

THE CEILING

Re-ranking: retrieve 20 cheap, keep the best 4

Retrieval is fast and shallow. Judging relevance properly is slow and expensive. So do both, in that order.

01 Hybrid search returns 20 candidates. Milliseconds.

02 Score each one for relevance, 0 to 10 — a cross-encoder, or the model itself.

03 Keep the best 4. Only those go into the prompt.

Why this wins

- Recall comes from the cheap wide net.
- Precision comes from the expensive judge.
- The prompt stays small, so generation stays cheap and focused.

Cost: one extra call per question. Usually the best value in the whole pipeline.

> This is the "k = 20" I told you not to do — the difference is that sixteen of them never reach the model.

PRESENTER NOTES

Come back to the top-k slide explicitly here, because this looks like a contradiction and it is not.

Wide and cheap first, narrow and expensive second. That pattern shows up all over search, and it is why the two-stage design keeps winning.

In the notebook we use the model as the judge, which is slower than a proper cross-encoder but needs nothing extra installed.

THE CEILING

The user's question is a bad search string

What they type

"and what about for part-timers?"

Meaningless alone. Retrieves nothing useful — the subject was two messages ago.

What you search for

"annual leave entitlement part-time employees"

One cheap call rewrites it using the conversation so far. Then you retrieve on that.

Also useful: generate two or three variations, retrieve for each, merge the results. Cheap recall.

PRESENTER NOTES

This is the fix for the second question in every conversation, which is where most chat-over-documents systems quietly fall apart.

The user says "and for part-timers?" and your retriever, which has no memory at all, searches for exactly that.

One small rewriting call before retrieval and follow-up questions start working. It is one of the highest-value additions in this deck.

أكاديمية سدايا
SDAIA Academy

SDAIA

PART FOUR · THE CEILING

Proving it works

"It looked right" is not a result. By 2:30 you will have a number.

PRESENTER NOTES

This last half hour is the part of the week I care most about, and it is the thing almost nobody does.

Everything we just added — hybrid, re-ranking, rewriting — is a guess until you can show that it moved a number.

أكاديمية سدايا
SDAIA Academy

EVALUATION

The golden question set

Ten questions **you already know the answers to**, written against your own documents, before you tune anything.

For each one, record which chunk *should* come back — or a phrase that must appear in whatever does.

Then: what fraction of the ten did retrieval get right? That is your hit rate.

How to write good ones

- Questions a real user would type, not questions your system will pass.
- Include the awkward ones: an ID, an acronym, an Arabic question against an English document.
- Include one whose answer **is not in the corpus** — "I don't know" is the correct output.
- Ten is enough to be useful today. A hundred is a proper suite.

> Write them **before** you tune. Otherwise you are marking your own homework.

PRESENTER NOTES

Ten questions and half an hour buys you the ability to answer "did that change help?" for the rest of the project's life.

The third bullet is the one people leave out, and it is the one that catches a system which confidently answers everything.

Write yours this afternoon against your own documents. That set is also what you show on Thursday when somebody asks whether your project actually works.

EVALUATION · CODE

Score it, three ways

```
GOLDEN = [
  {"q": "How many annual leave days for grade 11?",
   "must_contain": "Grade 11"},
  {"q": "What is SDAIA-F-CRS-201-01-V1?",
   "must_contain": "SDAIA-F-CRS-201"},
  # ... eight more, written by you
]

def evaluate(search_fn, k=5):
    hits = 0
    for item in GOLDEN:
        got = " ".join(r["text"]
                       for r in search_fn(item["q"], k))
        if item["must_contain"].lower() in got.lower():
            hits += 1
    return hits / len(GOLDEN)

for name, fn in [("naive", vector_search),
                 ("hybrid", hybrid_search),
                 ("reranked", reranked_search)]:
    print(name, evaluate(fn))
```

What comes out

RETRIEVER	HIT RATE
naive vector	6 / 10
hybrid	8 / 10
+ re-ranked	**9 / 10**

Now you can say which change helped, and by how much. That sentence is worth more than the code above it.

PRESENTER NOTES

Fourteen lines. That is the entire cost of being able to prove your system works.

The containment check is crude — it measures whether the right text was retrieved, not whether the answer was good. That is deliberate, because retrieval is the part you can fix today.

When your table looks like this one, screenshot it. That is the slide in your Thursday presentation that separates you from the pairs who say "it seemed to work".

أكاديمية سدايا
SDAIA Academy

SDAIA

BEFORE YOU BUILD

The four mistakes everyone makes

Chunks too big
A whole page per chunk. Retrieval "works", answers are vague, every call is expensive.

Top-k too high
Twenty chunks into the prompt. The answer is in there somewhere — and so is everything else.

No metadata
No source, no page, so no citations — and no way to tell a good answer from a confident one.

No evaluation
Changes made on vibes. Nobody can say whether last week's tweak helped or hurt.

> The fourth one is why the other three go unnoticed for months.

PRESENTER NOTES

I have seen all four of these in production systems built by good engineers, usually all four at once.

Read the last line and let it sit, because it is the argument for everything we did after two o'clock today.

LAB · NOTEBOOK 2

Now you build the whole thing

day2_retrieval.ipynb

- **Part A** — load, clean, read a scanned page with vision, chunk, embed, store, query, answer with citations.
- **Part B** — watch vector search miss an ID, add BM25, combine, re-rank, rewrite the query.
- **Part C** — write your golden set and score all three retrievers.

Two things will break

- **429s.** The batching and sleeping is already written. Do not remove it.
- **Re-running the store cell** duplicates your collection. Delete it first — there is a guard in the notebook.

Open the notebooks →

PRESENTER NOTES

Part A before lunch, Parts B and C after. If you finish A early, start pointing it at your own documents rather than waiting for me.

The two failures on the right will happen to somebody within ten minutes, and now they will know what it is.

And this afternoon we form project teams, so keep an eye on who is working well near you.

أكاديمية سدايا
SDAIA Academy

SDAIA
الهيئة السعودية للبيانات والذكاء الاصطناعي
Saudi Data & AI Authority

DAY 2

What today was

1 · If the answer is in a document, use RAG. If it needs a decision or an action, consider an agent — tomorrow.

2 · Five stages, and the first two decide whether the rest can work at all.

3 · Vector search cannot find an ID. Hybrid can. Re-ranking makes what comes back worth reading.

4 · **You have a number now.** Everything from here is measurable.

PRESENTER NOTES

Four things, and close on the fourth. You did not only build retrieval today, you built the ability to tell whether it is any good.

Tomorrow your retriever stops being the whole system and becomes one tool that an agent can choose to call. Say that clearly, because it is the payoff of the week.

Before anyone leaves: teams and project ideas, using the flipchart from Sunday.